I. Dokumentacja do lab. nr 1 - "Klasy oraz ich elementy składowe, metody klasy"

II. Imię i nazwisko - grupa ABC, semestr III

III. Przedmiot - "Programowanie obiektowe"

IV. Opis zadania do realizacji

Do zrealizowania były następujące zadania:

- Zaprojektuj klasę Book z polami title, author, available oraz metodami borrow() i returnBook().
  Pola powinny być private (enkapsulacja) — udostępnij je przez gettery, a stan available modyfikuj wyłącznie wewnątrz metod borrow() i returnBook(). Metoda borrow() powinna wypożyczać książkę tylko wtedy, gdy available == true (ustawia wtedy available = false i wypisuje komunikat); w przeciwnym razie powinna poinformować, że książka jest już wypożyczona. Metoda returnBook() powinna działać odwrotnie — zwraca książkę (available = true), a jeśli książka nie była wypożyczona, wypisuje stosowny komunikat. Dodaj konstruktor inicjalizujący title, author oraz ustawiający available na true, a także metodę toString() prezentującą stan obiektu. W metodzie main utwórz kilka obiektów Book, przetestuj wypożyczenie, próbę ponownego wypożyczenia tej samej książki oraz zwrot.
    - Zaimplementuj klasę BankAccount z prywatnym polem balance i metodami deposit(double) oraz withdraw(double) z walidacją danych.
      deposit(double kwota) powinna akceptować tylko kwoty dodatnie — dla kwoty ujemnej lub zerowej wypisz komunikat o błędzie i nie zmieniaj salda (por. przykład 2 — KontoBankowe). withdraw(double kwota) powinna dodatkowo sprawdzać, czy na koncie jest wystarczająco środków (kwota <= balance); jeśli nie, wypisz komunikat o braku środków i nie modyfikuj salda. Dodaj konstruktor przyjmujący saldo początkowe (z walidacją, że nie jest ujemne) oraz metodę getBalance(). Przetestuj w main scenariusze: poprawną wpłatę, wypłatę większą niż saldo oraz wpłatę ujemnej kwoty.
- Utwórz hierarchię Vehicle -> Car i pokaż w main, które pola/metody są dziedziczone.
  Klasa bazowa Vehicle powinna mieć wspólne pola (np. marka, rokProdukcji) oraz metodę, np. getDescription(), zwracającą podstawowy opis pojazdu (wzoruj się na przykładzie 4 — Zwierze/Pies/Kot). Klasa Car powinna dziedziczyć po Vehicle (extends Vehicle), wywoływać konstruktor nadklasy przez super(...) i dodawać własne pole, np. liczbaDrzwi. W main utwórz obiekt Car, wywołaj na nim zarówno metodę odziedziczoną z Vehicle, jak i metodę zdefiniowaną tylko w Car, i w komentarzach zaznacz, które elementy pochodzą z klasy bazowej, a które są specyficzne dla Car.
- Dodaj klasę ElectricCar, która przesłania metodę getDescription() i zademonstruj polimorfizm na tablicy Vehicle[].
  ElectricCar powinna dziedziczyć po Vehicle (ewentualnie po Car z zadania 3) i nadpisywać (@Override) metodę getDescription(), dodając informację specyficzną dla pojazdu elektrycznego, np. o zasięgu baterii (por. przykład 5 — Figura/Kolo/Prostokat/Trojkat). Utwórz tablicę typu Vehicle[]

zawierającą różne podklasy (Car, ElectricCar itd.) i w pętli for wywołaj na każdym elemencie getDescription() — zwróć uwagę, że mimo tego samego typu referencji (Vehicle) wywoływana jest metoda właściwa dla faktycznego typu obiektu (polimorfizm dynamiczny). Dodaj krótki komentarz wyjaśniający, dlaczego to jest przykład polimorfizmu, a nie zwykłego wywołania metody.

V. Technologie wykorzystane w zadaniu

- Java,

VI. Realizacja zadania

1. Kod Javy (lub Pythona)

W zadaniu 1 wykorzystano .

Kod wykorzystany do rozwiązania zadania (zadań):

```
public class Book {
 private String title;
 private String author;
 private boolean avaiable;

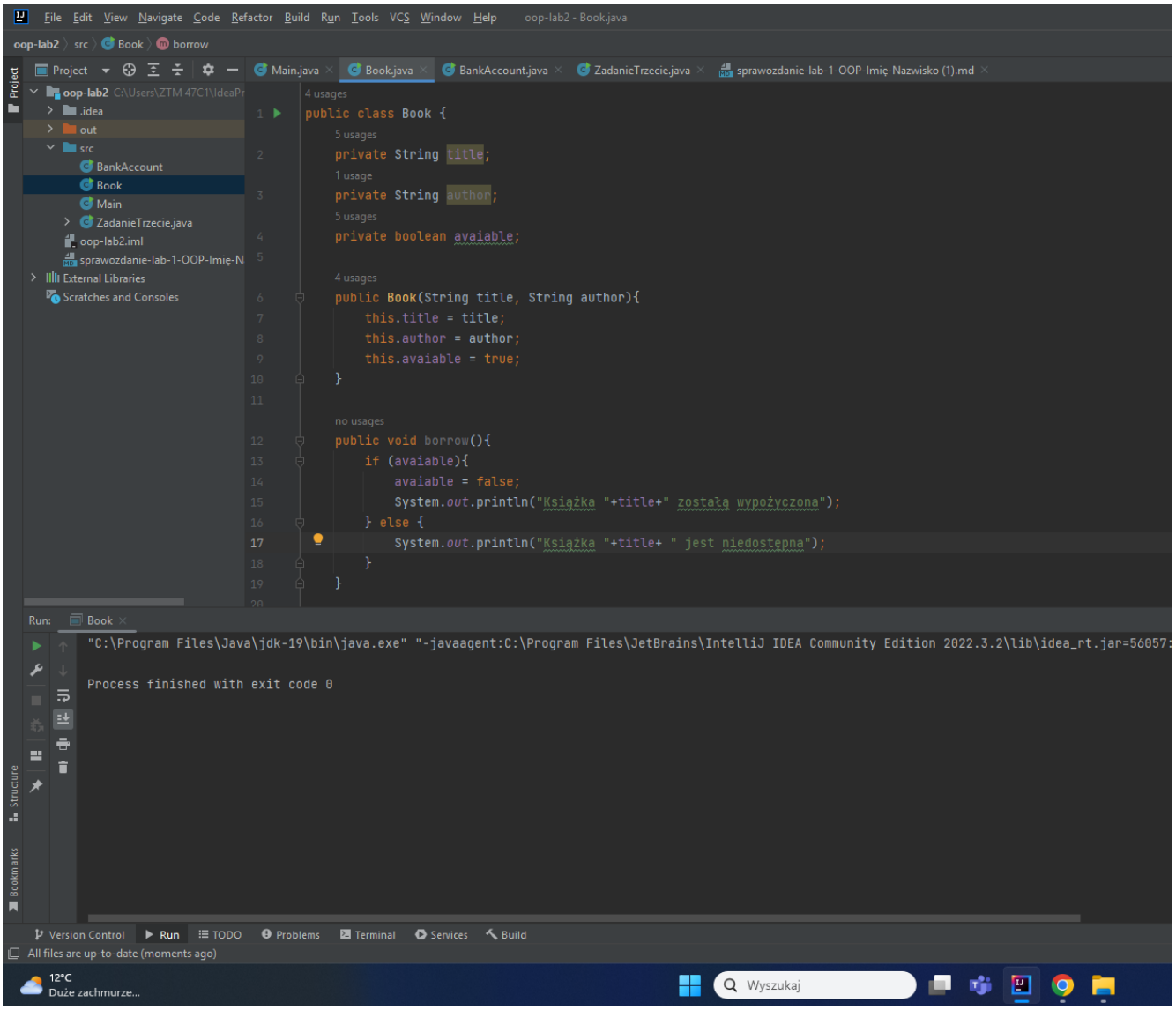
 public Book(String title, String author){
     this.title = title;
     this.author = author;
     this.avaiable = true;
 }

 public void borrow(){
     if (avaiable){
         avaiable = false;
         System.out.println("Książka "+title+" została wypożyczona");
     } else {
         System.out.println("Książka "+title+ " jest niedostępna");
     }
 }

 public void returnBook(){
     if (!avaiable){
         avaiable = true;
         System.out.println("Książka "+title+" została zwrócona");
     }else{
         System.out.println("Książka"+title+" nie była wypożyczona");
     }
 }
 public static void main(String[] args) {

 }
}
```

2. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



1. Kod Javy (lub Pythona)

W zadaniu 2 wykorzystano .

Kod wykorzystany do rozwiązania zadania (zadań):

```
public class BankAccount {
 private double balance;

 public BankAccount(double initialBalance){
     if (initialBalance < 0){
         System.out.println("Błąd saldo nie może być ujemne.");
     }else{
         this.balance = initialBalance;
     }
 }

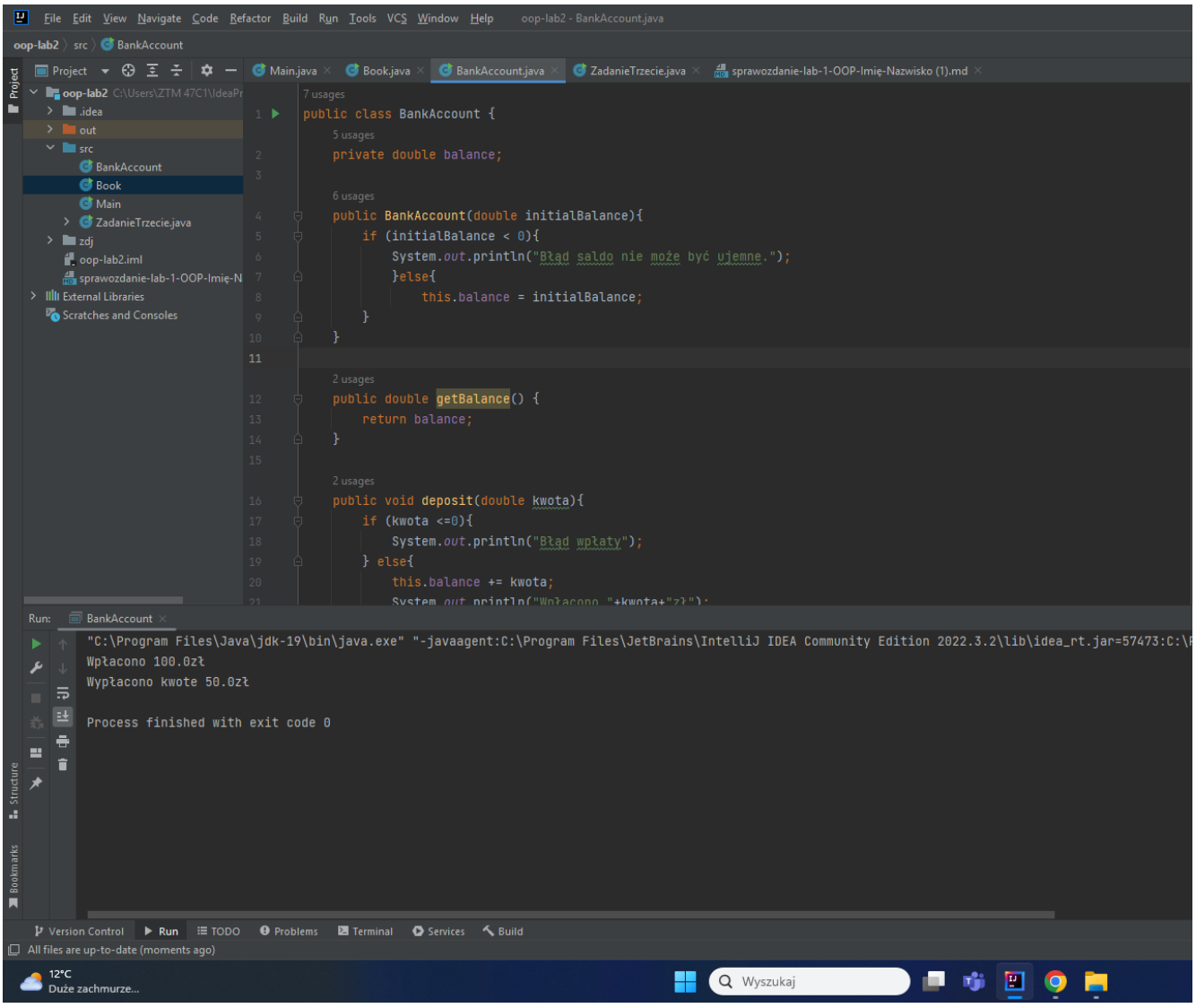
 public double getBalance() {
     return balance;
 }

 public void deposit(double kwota){
```

```
        if (kwota <=0){
            System.out.println("Błąd wpłaty");
        } else{
            this.balance += kwota;
            System.out.println("Wpłacono "+kwota+"zł");
        }
    }

    public void withdraw(double kwota){
        if (kwota <= balance){
            System.out.println("Wypłacono kwote "+kwota+"zł");
            this.balance =- kwota;
        } else{
            System.out.println("Brak środków na koncie");
        }
    }
    public static void main(String[] args) {
        BankAccount account = new BankAccount(0);
        account.deposit(100);
        account.withdraw(50);
        account.getBalance();
    }
}
```

2. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



1. Kod Javy (lub Pythona)

W zadaniu 3 i 4 wykorzystano .

Kod wykorzystany do rozwiązania zadania (zadań):

```
class Vehicle {
    protected String marka;
    protected int rokProdukcji;

    Vehicle(String marka, int rokProdukcji){
        this.marka = marka;
        this.rokProdukcji=rokProdukcji;

    }

    void getDescription(){
        System.out.println("Pojazd: "+ marka+ ", rok produkcji: "+ rokProdukcji);
    }
}

class Car extends Vehicle{
    int liczbaDrzwi;

    Car(String marka, int rokProdukcji, int liczbaDrzwi) {
        super(marka, rokProdukcji);
        this.liczbaDrzwi=liczbaDrzwi;
    }

    void opis() {
        System.out.println("Pojazd: "+ marka+ ", rok produkcji: "+ rokProdukcji+ ", Liczba drzwi: "+l
    }

}

class ElectricCar extends Vehicle {
    private int zasiegBaterii;

    public ElectricCar(String marka, int rokProdukcji, int liczbaDrzwi, int zasiegBaterii){
        super (marka, rokProdukcji);
        this.zasiegBaterii = zasiegBaterii;
    }

    @Override
    void getDescription(){
        System.out.println("Pojazd: "+ marka+ ", rok produkcji: "+ rokProdukcji+", zasięg baterii"+za
    }
}

public class ZadanieTrzecie {
    public static void main(String[] args) {
        Car car1 = new Car("BMW", 1999, 5);
        car1.getDescription();
        car1.opis();
    }
}
```

2. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:

